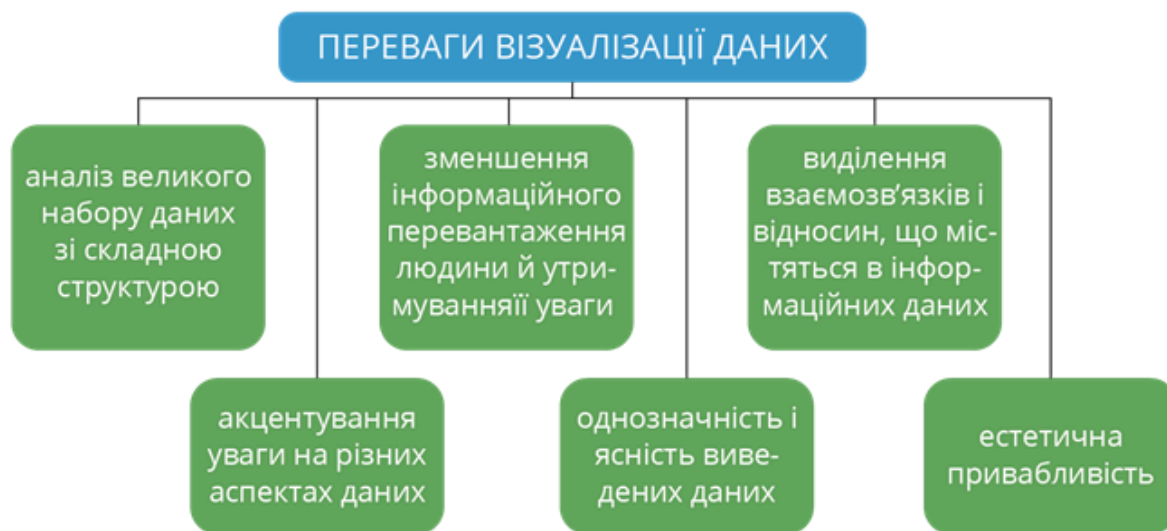


Конспект лекції

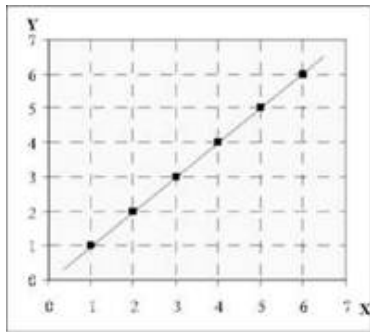
▼ Призначення і важливість візуального аналізу та візуалізації даних

Візуалізація даних потрібна, з одного боку, для аналізу даних, для виявлення залежностей, відбору фіч, порівняння якості моделей та підбору гіперпараметрів. А з іншого боку, це інструмент для візуалізації результатів аналізу. Від якості проведеного аналізу на пряму залежать якість препроцесінгу і, відповідно, показники моделі

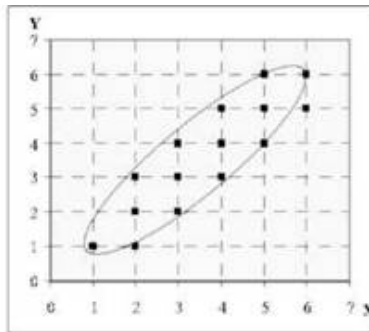
Головна перевага візуалізації – практично повна відсутність необхідності в спеціальній підготовці користувача. За допомогою візуалізації ознайомитися з інформацією дуже легко, досить лише на неї подивитися. Візуалізація даних може бути представлена у вигляді: графіків, схем, гістограм, діаграм тощо.



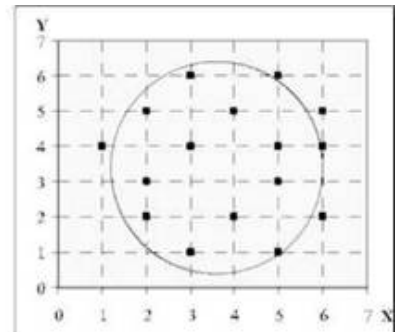
Візуальний аналіз допомагає побачити те, що не видно з колонки цифр (розподіл значень, наявність викидів, кореляція між колонками...)



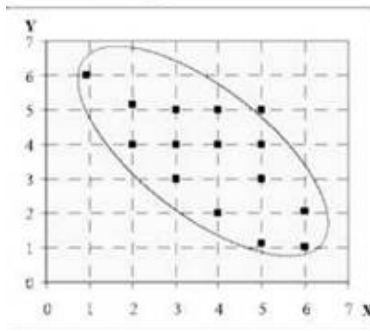
а) $r_{xy} = +1,00$



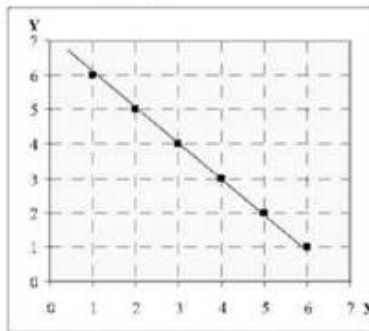
б) $r_{xy} \approx +0,88$



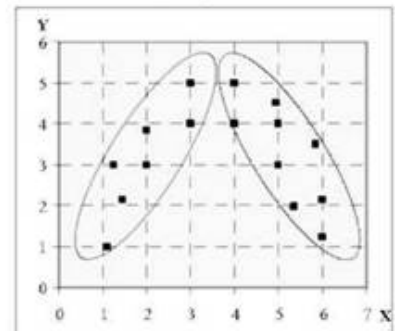
в) $r_{xy} \approx 0$



г) $r_{xy} \approx -0,60$

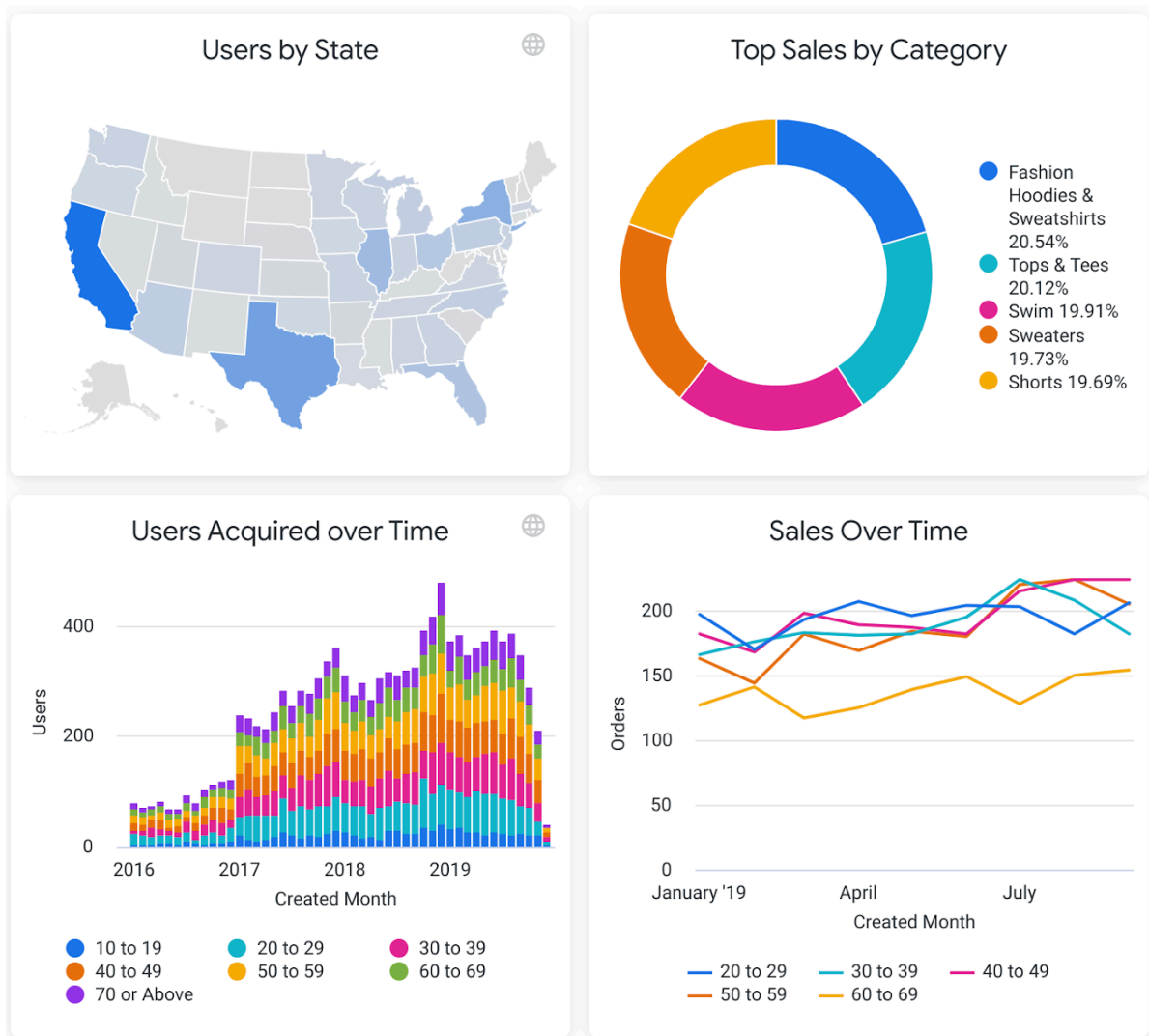


д) $r_{xy} = -1,00$



е)

Якщо мова йде про велику розмірність і складності вихідних даних, кошти візуалізації забезпечують їхнє різке зменшення, конденсуючи, можливо, мільйони записів даних у прості, легкі для розуміння й маніпулювання показники. Такі показники називають візуальним або графічним способом показу інформації.



▼ Первинний аналіз даних

Для початку імпортуємо бібліотеки, завантажимо датасет, проведемо первинний аналіз даних

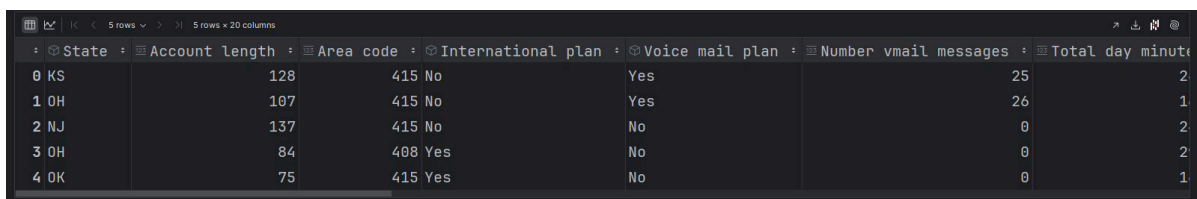
```
# Matplotlib forms basis for visualization in Python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
# We will use the Seaborn library
import seaborn as sns

sns.set()
```

```
# Graphics in SVG format are more sharp and legible
%config InlineBackend.figure_format = 'svg'
```

```
df = pd.read_csv("telecom_churn.csv")
```

```
df.head()
```



	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes
0	KS	128	415	No	Yes	25	2.0
1	OH	107	415	No	Yes	26	1.0
2	NJ	137	415	No	No	0	2.0
3	OH	84	408	Yes	No	0	2.0
4	OK	75	415	Yes	No	0	1.0

Опис колонок зазвичай є в описі датасету. Приведемо його тут

Here is the description of our features:

Name	Description	Value Type	Statistical Type
State	State abbreviation (like KS = Kansas)	String	Categorical
Account length	How long the client has been with the company	Numerical	Quantitative
Area code	Phone number prefix	Numerical	Categorical
International plan	International plan (on/off)	String, "Yes"/"No"	Categorical/Binary
Voice mail plan	Voicemail (on/off)	String, "Yes"/"No"	Categorical/Binary
Number vmail messages	Number of voicemail messages	Numerical	Quantitative
Total day minutes	Total duration of daytime calls	Numerical	Quantitative
Total day calls	Total number of daytime calls	Numerical	Quantitative
Total day charge	Total charge for daytime services	Numerical	Quantitative
Total eve minutes	Total duration of evening calls	Numerical	Quantitative
Total eve calls	Total number of evening calls	Numerical	Quantitative
Total eve charge	Total charge for evening services	Numerical	Quantitative
Total night minutes	Total duration of nighttime calls	Numerical	Quantitative
Total night calls	Total number of nighttime calls	Numerical	Quantitative
Total night charge	Total charge for nighttime services	Numerical	Quantitative
Total intl minutes	Total duration of international calls	Numerical	Quantitative
Total intl calls	Total number of international calls	Numerical	Quantitative
Total intl charge	Total charge for international calls	Numerical	Quantitative
Customer service calls	Number of calls to customer service	Numerical	Categorical/Ordinal

Перевіряємо чи вірно завантажено датасет (типи даних) і чи є пропуски в даних

```
df.info()
```

```
Executed at 2024.05.06 11:07:59 in 63ms
```

#	Column	Non-Null Count	Dtype
0	State	3333 non-null	object
1	Account length	3333 non-null	int64
2	Area code	3333 non-null	int64
3	International plan	3333 non-null	object
4	Voice mail plan	3333 non-null	object
5	Number vmail messages	3333 non-null	int64
6	Total day minutes	3333 non-null	float64
7	Total day calls	3333 non-null	int64
8	Total day charge	3333 non-null	float64
9	Total eve minutes	3333 non-null	float64
10	Total eve calls	3333 non-null	int64
11	Total eve charge	3333 non-null	float64
12	Total night minutes	3333 non-null	float64
13	Total night calls	3333 non-null	int64
14	Total night charge	3333 non-null	float64
15	Total intl minutes	3333 non-null	float64
16	Total intl calls	3333 non-null	int64
17	Total intl charge	3333 non-null	float64
18	Customer service calls	3333 non-null	int64
19	Churn	3333 non-null	bool

```
dtypes: bool(1), float64(8), int64(8), object(3)
```

```
memory usage: 498.1+ KB
```

Бачимо, що типи колонок відповідають опису і нульові значення відсутні.
Якщо бачимо, що в окремих колонках присутні нульові значення -

необхідно буде додаково пропрацювати ці колонки і рядки (видалити або заповнити дефолтними значеннями)

▼ Модулі matplotlib, seaborn, plotly

▼ Matplotlib

Ця бібліотека найчастіше використовується як бекенд для Pandas. Щоб встановити цю бібліотеку треба виконати в терміналі команди:

```
python -m pip install -U pip
python -m pip install -U matplotlib
```

▼ Seaborn

Бібліотека використовує matplotlib як бекенд і додає широкі можливості по роботі із графіками (агрегація даних, нанесення на графік додаткових елементів, групування графіків в один фрейм, тощо).

Для встановлення бібліотеки треба виконати в терміналі команди:

```
pip install seaborn
```

▼ Plotly

Бібліотека plotly, на відміну від двох попередених бібліотек, дозволяє створювати інтерактивні зображення, на яких можна наводити на точку на графіку і отримувати інформацію про значення, змінювати параметри графіка, або зробити анімацію.

Для встановлення бібліотеки треба виконати в терміналі команди:

```
pip install plotly
# pip install plotly==5.22.0 # якщо потрібна конкретна версія
```

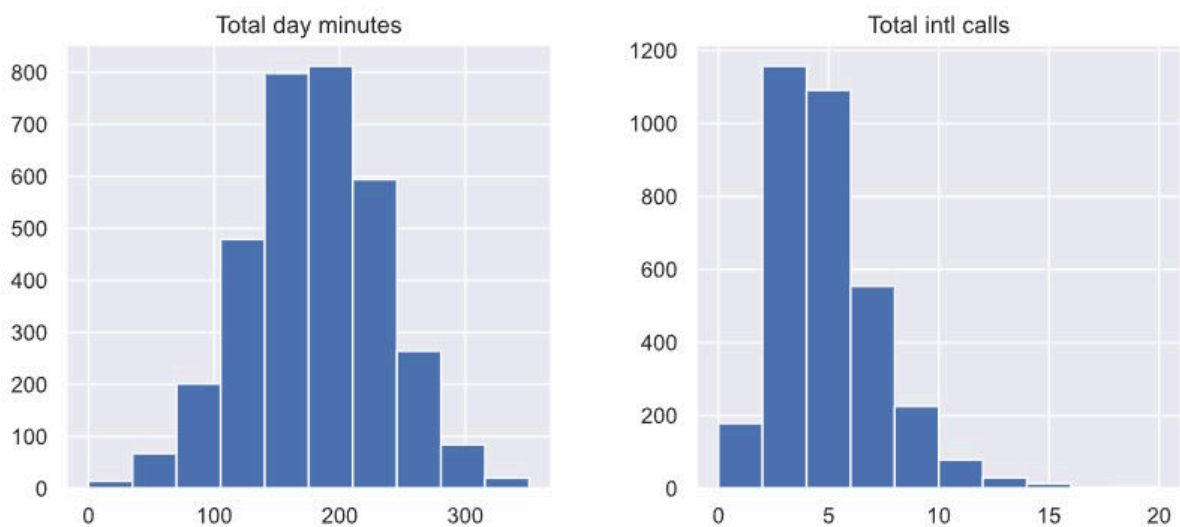
Крім цих бібліотек, існують і інші бібліотеки з різною функціональністю. Частина має вузький специфічний функціонал, наприклад **Plotnine** (**ggplot2**), **geoplotlib**, **MultiPolyline (Folium)**. Частина є портованими

версіями відомих бібліотек для R: **Glean**, **Plotnine**. Частина використовує в якості бекенду не `matplotlib`, а інші бібліотеки: **Bokeh**, **pygal**, **missingno**, **leather**, **Steamgraph (Altair)**.

▼ Побудова простих графіків однієї змінної

При аналізі графіків однієї змінної зазвичай перевіряють розподіл значень

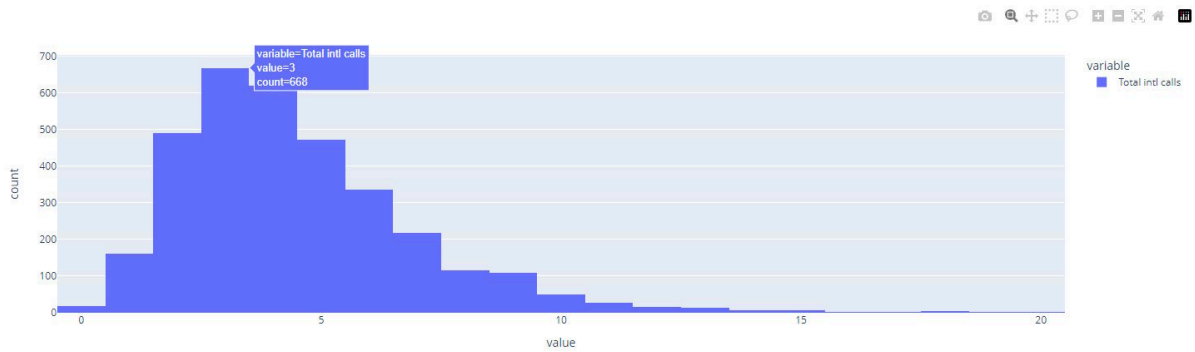
```
features = ["Total day minutes", "Total intl calls"]
df[features].hist(figsize=(10, 4));
```



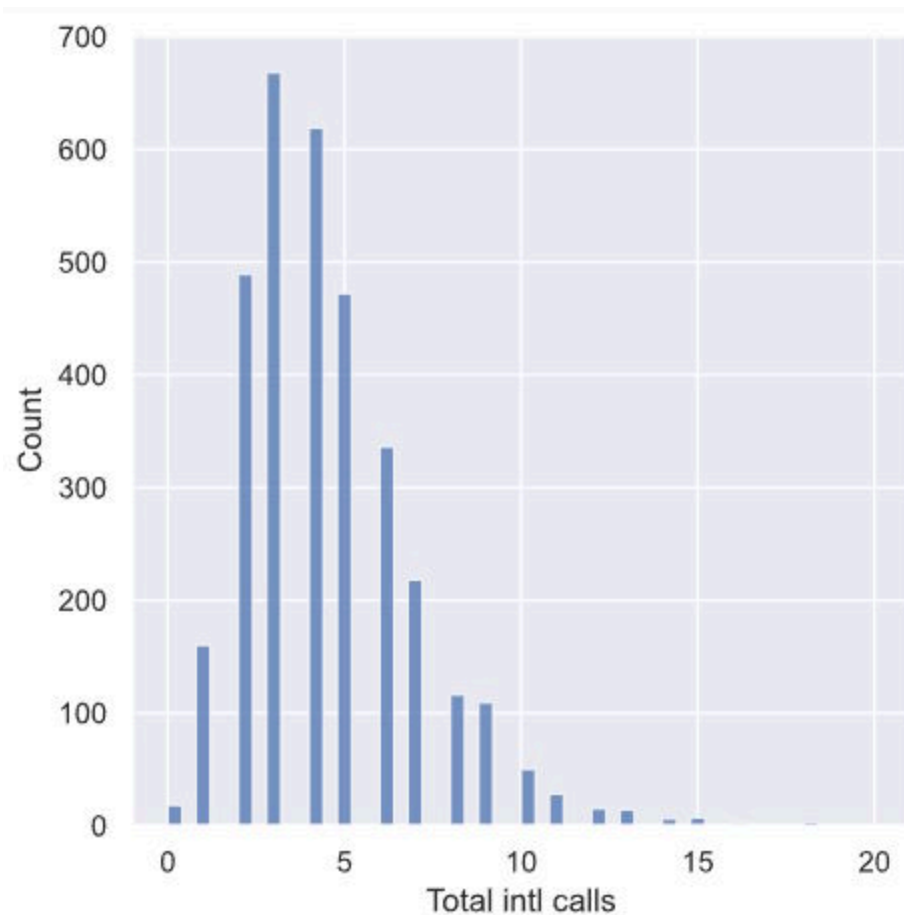
Бажано при візуальному аналізі даних завжди залишати коментарі чому саме ця колонка була обрана для візуалізації і які висновки ми робимо на основі побаченого.

Побудуємо правий графік за допомогою бібліотек `plotly` і `seaborn`

```
import plotly.express as px
fig = px.histogram(df["Total intl calls"], x=["Total intl calls"])
fig.show()
```

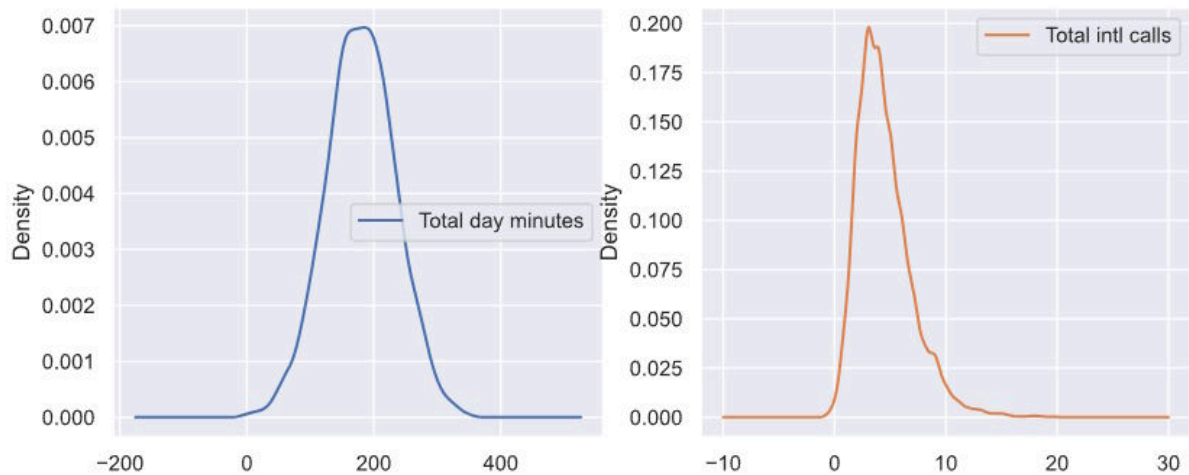


```
sns.displot(df["Total intl calls"]);
```



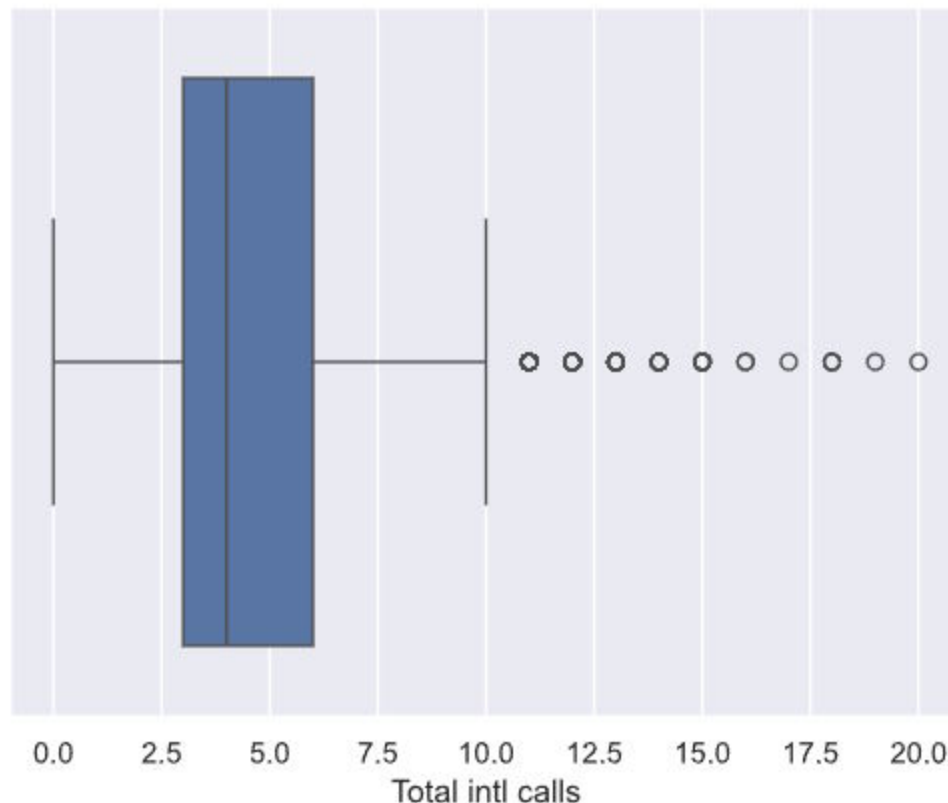
Гістограми можуть усереднювати значення в межах діапазону. Щоб побачити значення кожної точки можемо побудувати графік у звичному нам вигляді. Такі графіки називають density plots або точніше Kernel Density Plots.

```
df[features].plot(  
    kind="density", subplots=True, layout=(1, 2), sharex=False,  
    figsize=(10, 4)  
);
```



Іншим цікавим представленням розподілу значень змінної є boxplot, або "ящик із вусами"

```
sns.boxplot(x="Total intl calls", data=df);
```



Цей графік показує міжквартильний або інтерквартильний розмах (interquartile range, IQR). Лівий і правий бік "ящика" - це значення для 25-го перцентиля (першого або нижнього квантиля) і 75-го перцентиля (третього або верхнього квантиля). Вертикальна лінія всередині відповідає 50-му перцентилю (медіана або другий квантиль)

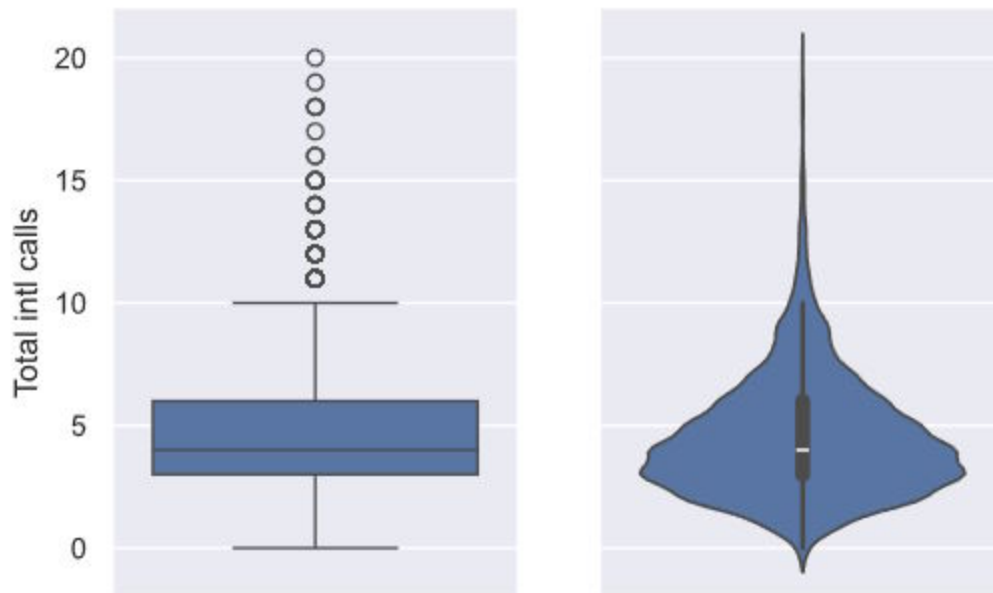
"Вуса" показують безпосередньо інтерквартильний розмах. Вони розраховуються за формулою $(Q1 - 1.5 \cdot IQR; Q3 + 1.5 \cdot IQR)$, де $Q1$, $Q3$ - значення нижнього і верхнього квантилів, $IQR = Q3 - Q1$

Точки за межами "вус" будемо вважати аутлейерами. До них повернемось нижче

Ще одним цікавим представленням інтерквартильного розмаху є violin plot. Крім безпосередньо інтервалу він показує і розподіл значення величини

```
_, axes = plt.subplots(1, 2, sharey=True, figsize=(6, 4))
sns.boxplot(data=df["Total intl calls"], ax=axes[0])
```

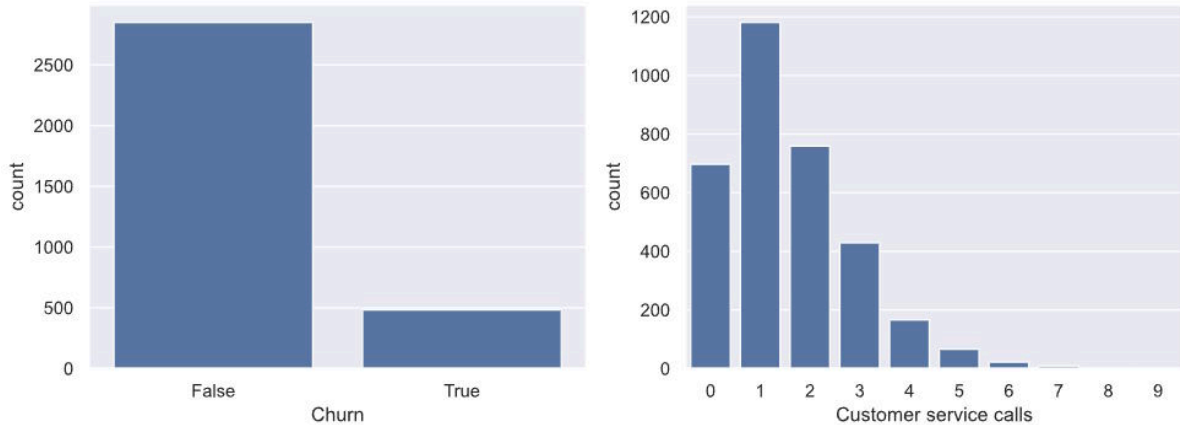
```
sns.violinplot(data=df["Total intl calls"], ax=axes[1]);
```



Як бачимо, він також візуалізує квантілі, медіанне значення. Тобто ці два графіка є альтернативними представленнями і можуть бути застосовані окремо, або в парі.

Для аналізу категоріальних даних або таргета використовують bar plot. Хоч він і виглядає майже так само, як і гістограма, відмінність в суті. При побудові гістограми ми розбиваємо інтервали на задану кількість ділянок і рахуємо кількість значень, що потрапили в цю ділянку. При побудові bar plot ми показуємо кількість кожного значення в окремій категорії

```
_, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))  
  
sns.countplot(x="Churn", data=df, ax=axes[0])  
sns.countplot(x="Customer service calls", data=df, ax=axes  
[1]);
```



Підсумовуючи даний розділ, слід зазначити, що основна ідея дослідження кожної окремої колонки – виявити закономірності розподілу даних, наявність аутлейерів, проаналізувати чому значення колонки мають саме такий візуальний розподіл. Це допоможе в подальшому визначити кроки препроцесінгу даних, визначити колонки, для яких слід перевірити за метриками нормальності розподілу.

▼ Побудова графіків двох і більше змінних

На відміну від дослідження розподілу однієї змінної (колонки даних), попарний аналіз дає можливість побачити кореляції між значеннями колонок. Це дуже важливий крок, оскільки наявність пари колонок, що корелюють між собою може значно погіршити показники моделей в майбутньому.

Побудуємо heatmap. Для побудови цього графіка розраховується кореляція між парою змінних, і на отриманих даних будується дана карта.

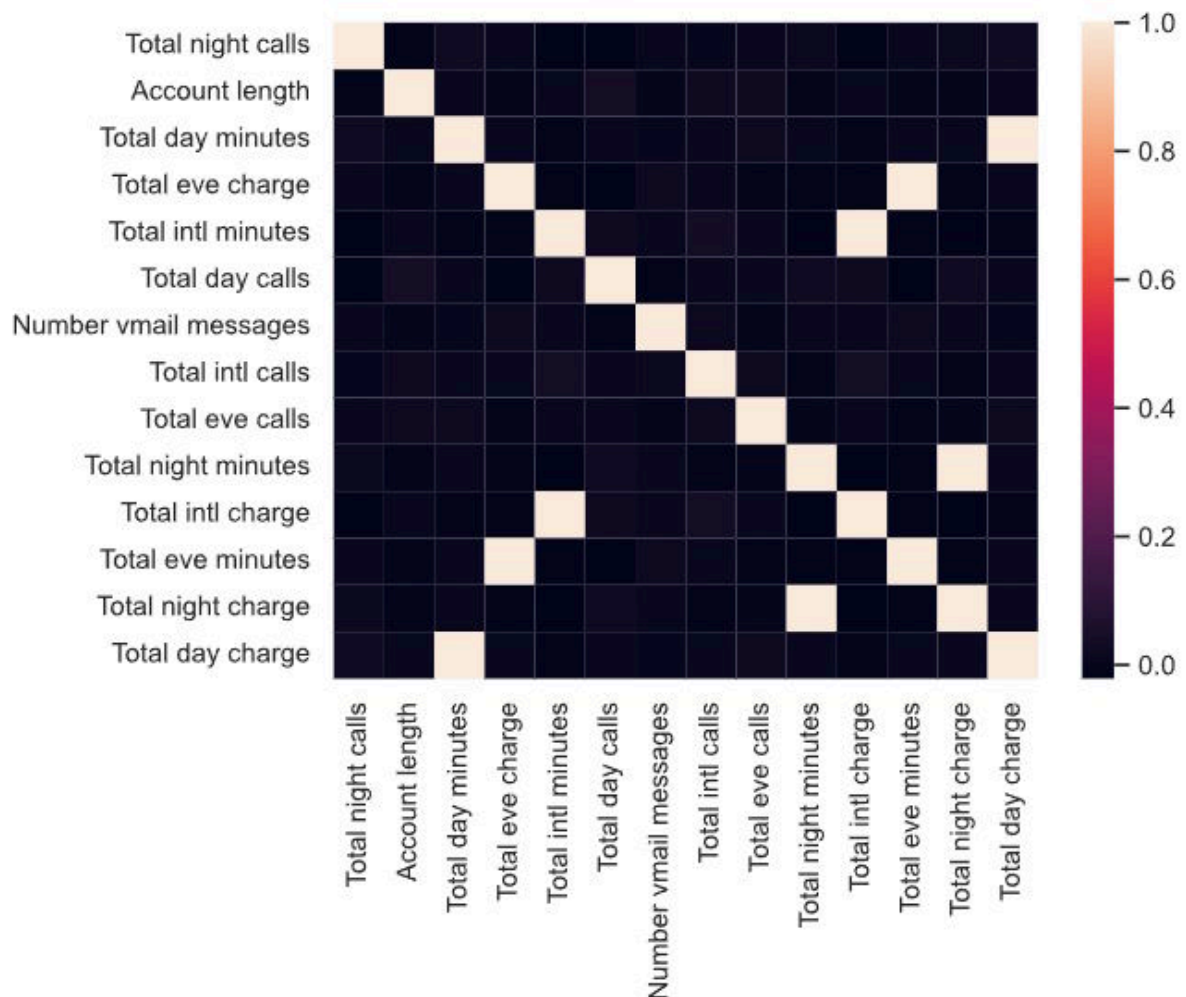
```
# Drop non-numerical variables
numerical = list(
    set(df.columns)
    - set(
        [
            "State",
            "International plan",
            "Voice mail plan",
            "Area code",
```

```

        "Churn",
        "Customer service calls",
    ]
)
)

# Calculate and plot
corr_matrix = df[numerical].corr()
sns.heatmap(corr_matrix);

```



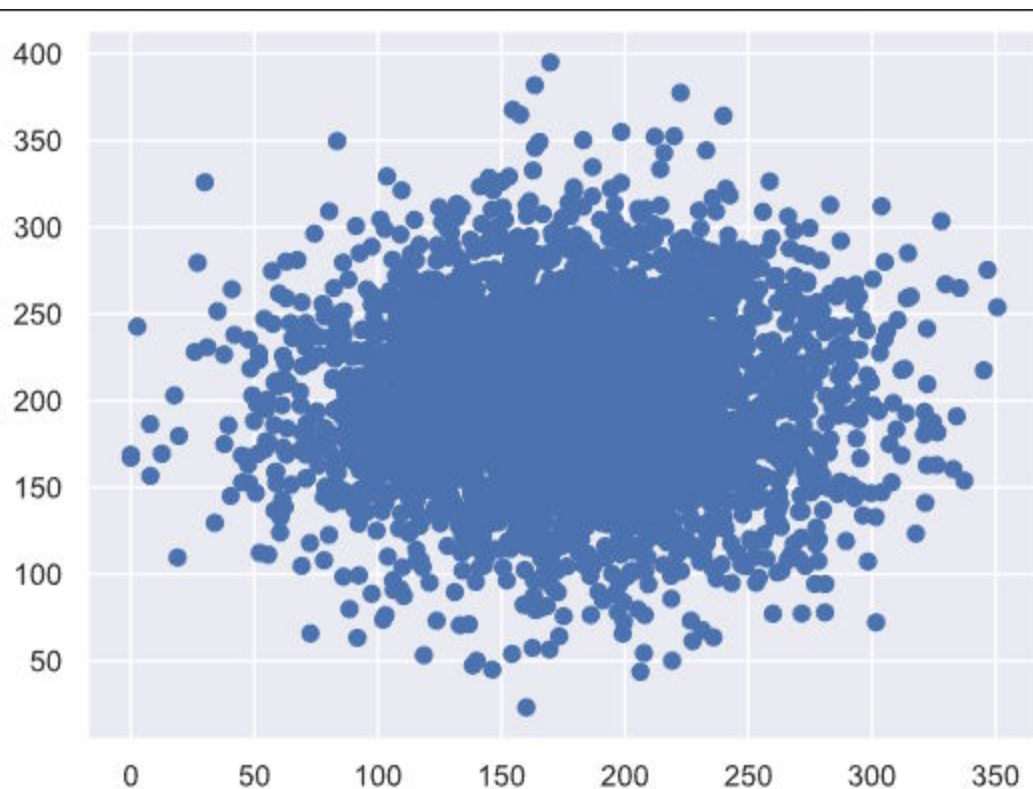
Значення кореляції може змінюватись від -1 до 1. -1 означає, що пара колонок мають повну від'ємну кореляцію (умовно можна значення однієї колонки домножити на від'ємну не нульову константу і отримати значення

іншої колонки). 1 означає повну позитивну кореляцію пари колонок. 0 - дві колонки є взаємно незалежні. Якщо дві колонки корелюють із коефіцієнтом більше ніж 0.8 за модулем, з якої пари слід залишити лише одну колонку, а другу видалити.

Як бачимо із графіку, є чотири пари колонок, що мають високу кореляцію. Це не дивно, оскільки в кожній пар є кількість хвилин у відповідну пору доби і плата, нарахована за ці хвилини із розрахунку X\$/хв.

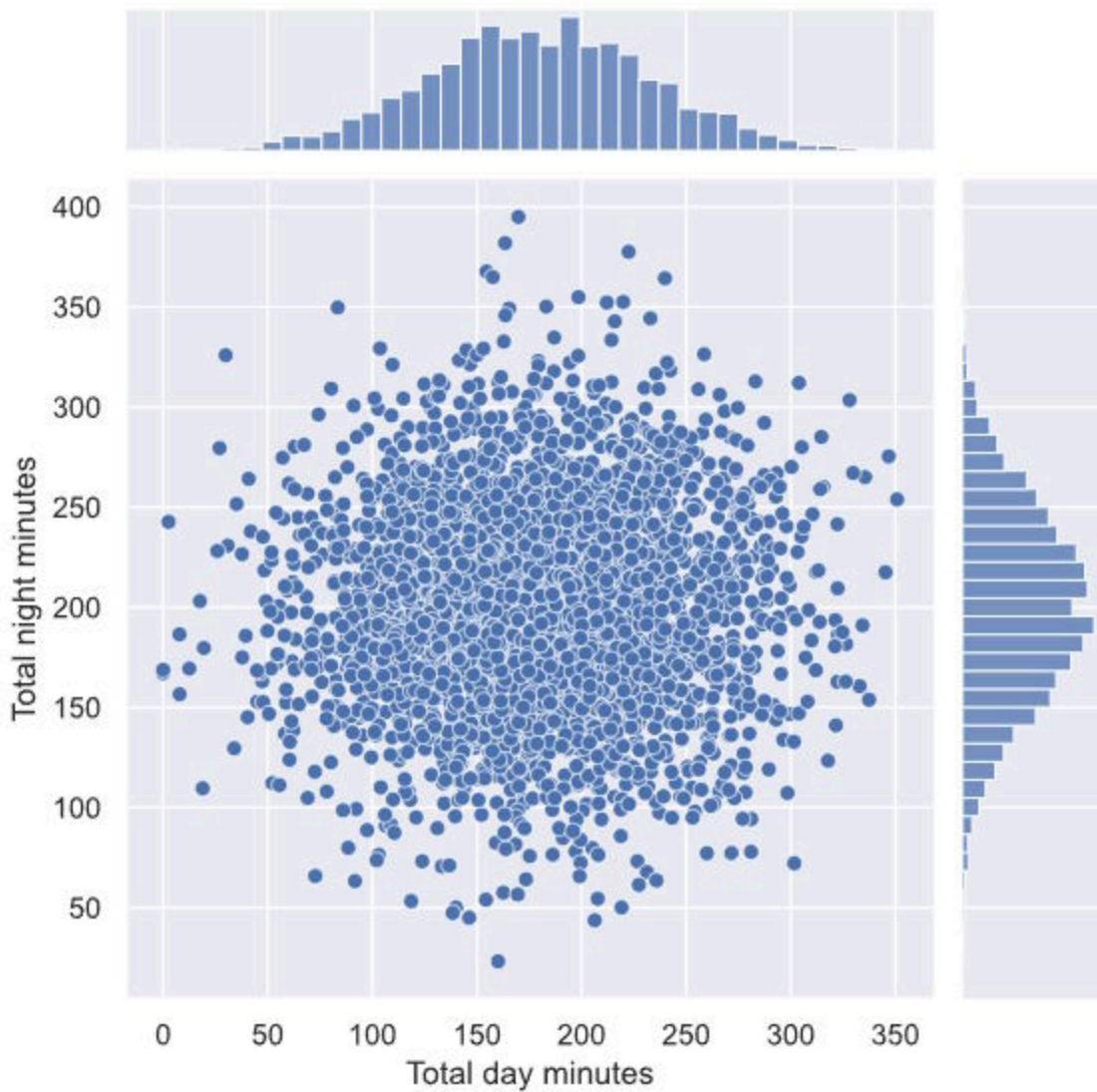
Scatter plot показує розподіл точок в площині. Значення однієї колонки - перша координата, значення другої колонки - друга координата

```
plt.scatter(df["Total day minutes"], df["Total night minutes"]);
```



Це нецікавий розподіл, оскільки ми бачимо утворений точками еліпс, а це говорить про те, що кожна із колонок має близький до нормального розподіл, і колонки не залежать одна від одної. Перевіримо це, побудувавши `jointplot`.

```
sns.jointplot(x="Total day minutes", y="Total night minutes", data=df, kind="scatter");
```

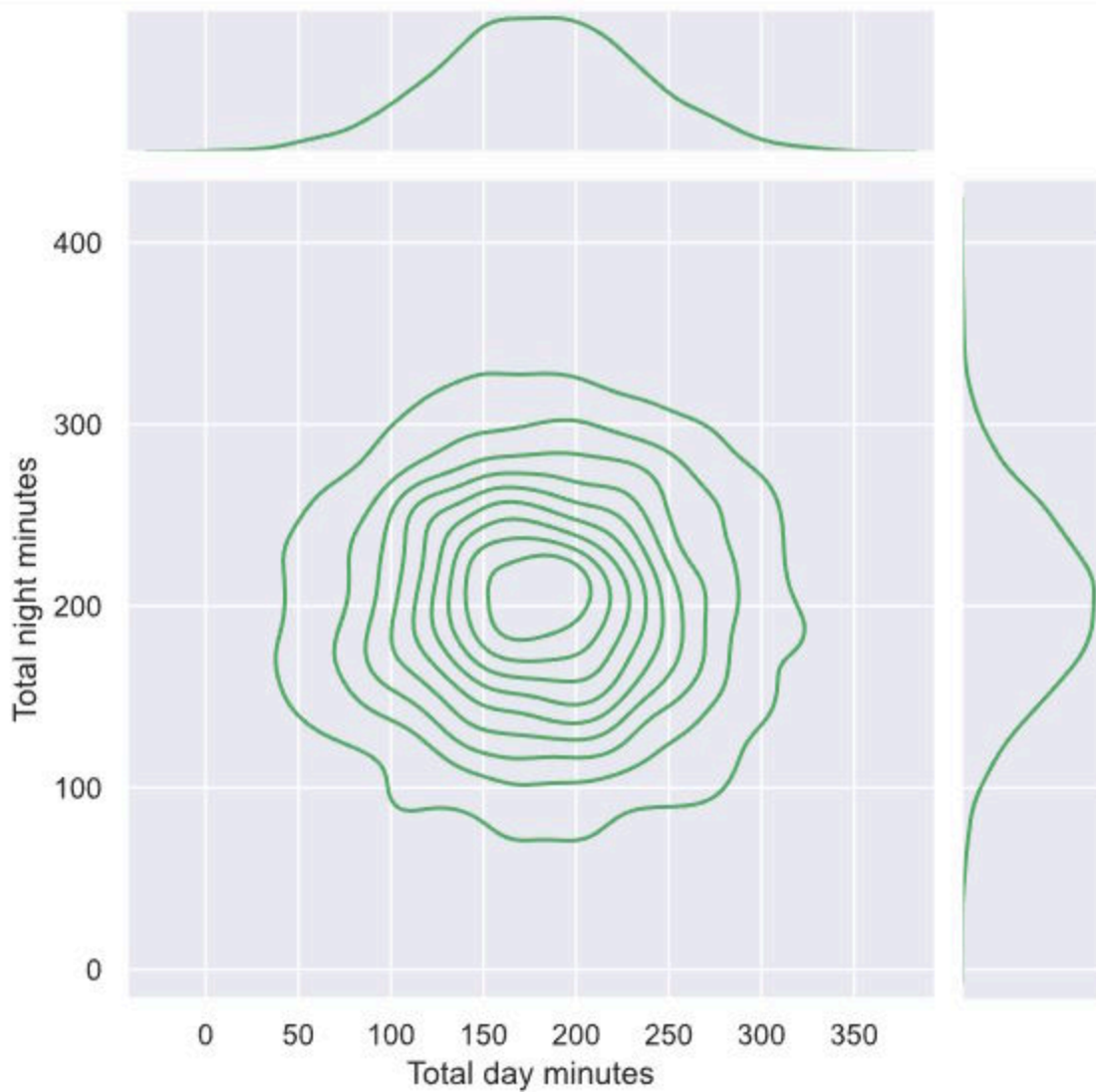


До попереднього графіка вздовж кожної вісі додали розподіл значень кожної колонки. І, дійсно ми бачимо, що розподіл кожної з колонок близький до нормального

Трохи інше представлення того ж графіка

```
sns.jointplot(x="Total day minutes", y="Total night minutes", data=df, kind="scatter");
```

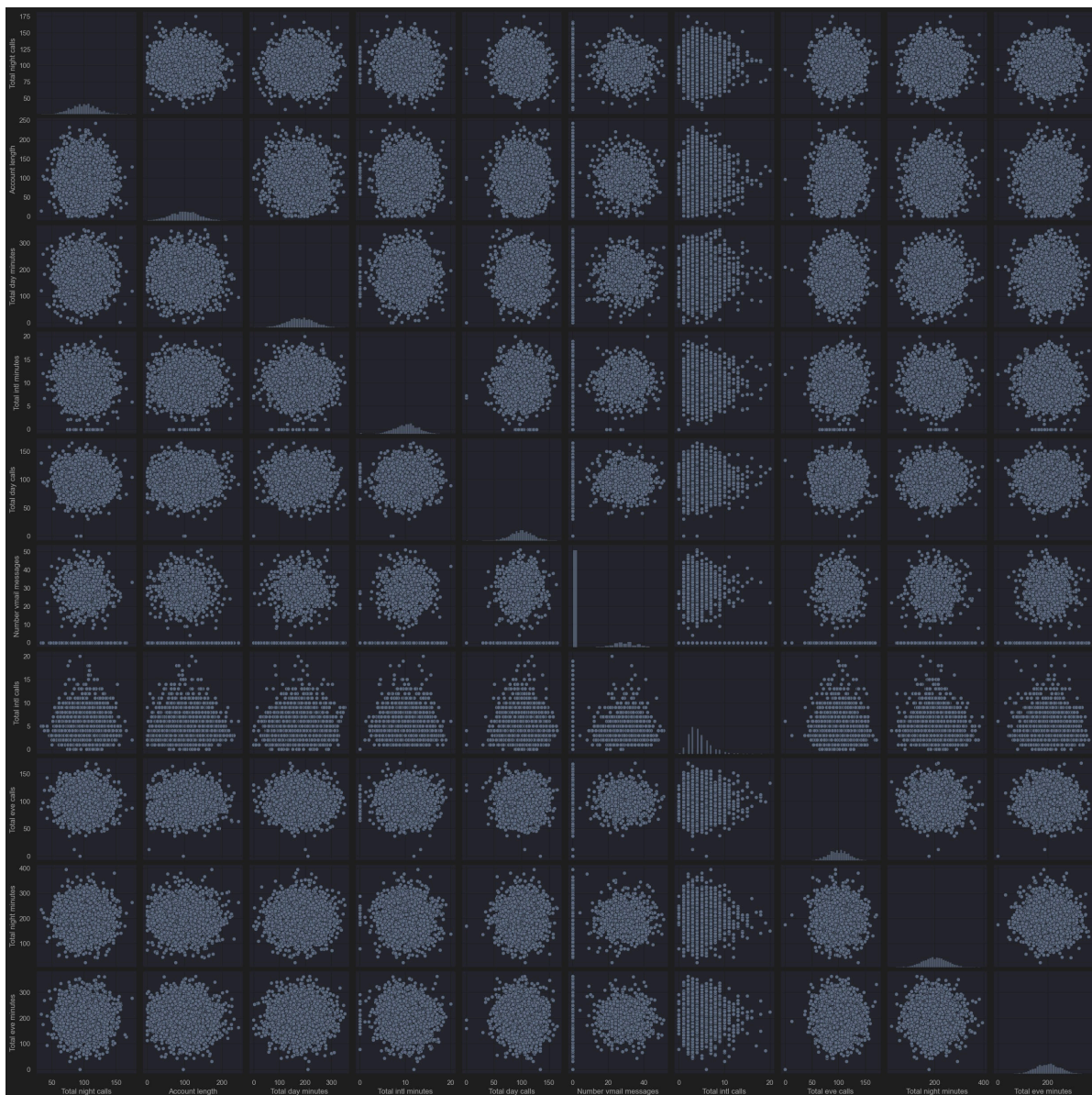
```
s", data=df, kind="kde", color="g");
```



Бокові графіки представлені у вигляді кривих, а центральний - геодезичні криві, або криві одного рівня.

Іноді, нам може бути потрібно побудувати щось середнє між heatmap і scatter plot. Такий графік називають `pair plot`.

```
# `pairplot()` may become very slow with the SVG format
%config InlineBackend.figure_format = 'png'
sns.pairplot(df[numerical]);
```



Цей вид графіку дозволяє побачити попарні scatter plot. Головна діагональ показує гістограму розподілу значень колонки. В даному випадку майже всі клітинки містять еліпси точок. В загальному випадку можна побачити витягнуті еліпси (кореляція між колонками), фігури, що схожі на профіль крила літака (колонки мають залежність, але вона виражена функцією, наприклад логарифмом, ступеневою функцією, тощо), полоси говорять, що відповідна колонка може бути розглянута і як категоріальна змінна. Іноді зустрічаються фігури, схожі на трикутники.

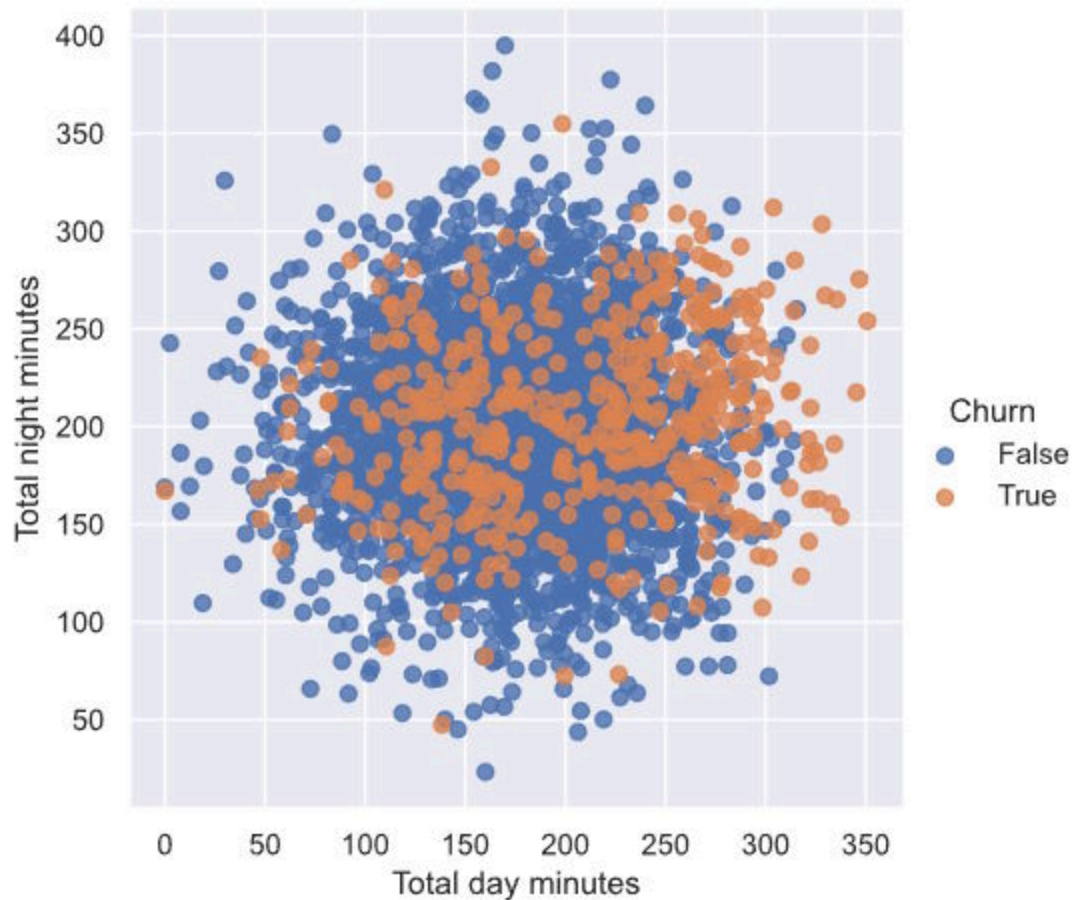
Попарний аналіз колонок дозволяє виявити приховані залежності між колонками. А це - частина препроцесінгу даних. Ми знаємо які колонки треба викинути, які мають залежність, що може бути описана певною функцією, і це також причина видалити одну з колонок, які колонки можна спробувати описати як категоріальні

▼ Аналіз графіків (перевірка кореляції із таргетом, виокремлення аутлейерів)

Робота з аутлейерами зводиться до виявлення тих колонок, в яких вони присутні, і намагання оцінити вплив викидів на точність моделі. На даному етапі ми лише фіксуємо той факт, що в колонці є викиди і за яким значенням ми можемо їх відкинути. В подальшому ми можемо спробувати, навіть, будувати дві різні моделі: модель на даних без викидів і модель лише на викидах. І маємо порівняти результат на моделі, яка тренувалась на даних із викидами.

Візуалізація стає цікавішою коли ми спробуємо додати таргет до побудови графіків. Наприклад до scatter plot

```
sns.lmplot(x="Total day minutes", y="Total night minutes",  
data=df, hue="Churn", fit_reg=False);
```



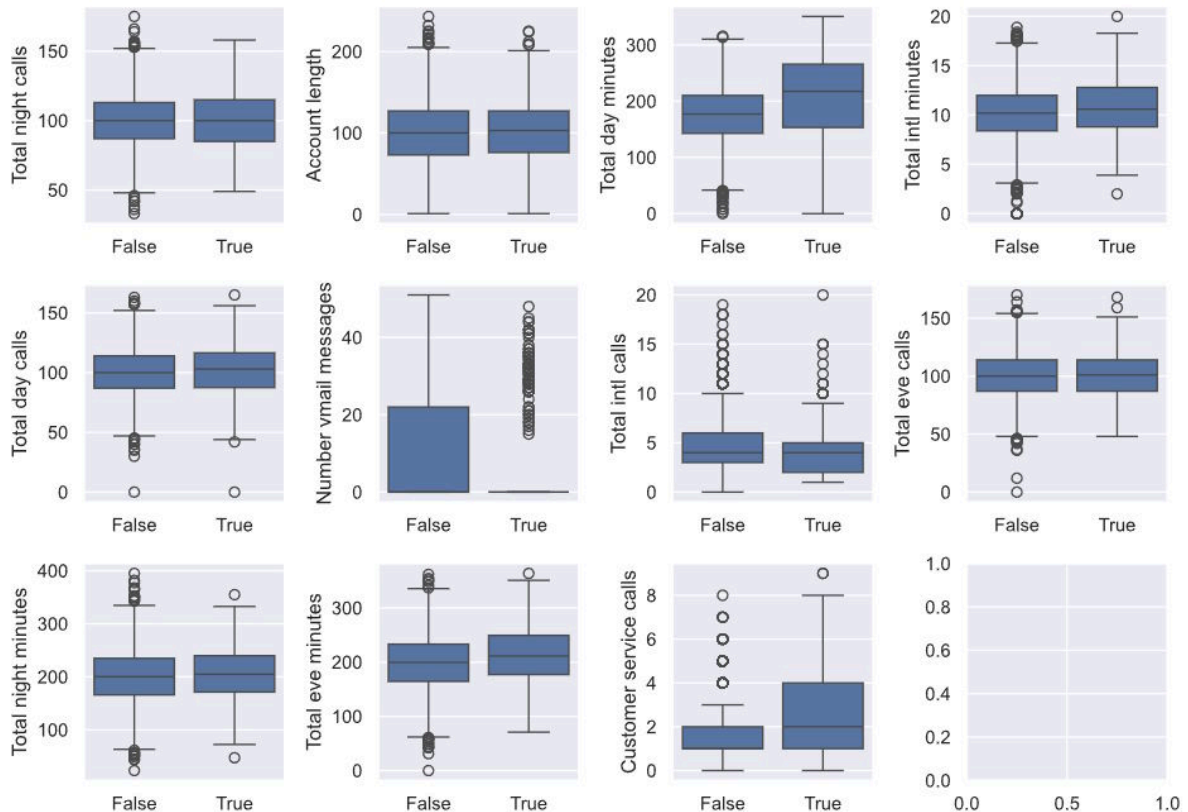
В цій парі ми не бачимо явного скупчення жовтих чи синіх точок. То ж можемо зробити висновок, що ці дві колонки в явному вигляді не визначають таргет.

Побудуємо декілька "ящиків із вусами", але в розрізі таргета. Тобто в кожній парі графіків один буде відповідати одному значенню класа, а другий - іншому.

```
# Sometimes you can analyze an ordinal variable just as numerical one
numerical.append("Customer service calls")

fig, axes = plt.subplots(nrows=3, ncols=4, figsize=(10, 7))
for idx, feat in enumerate(numerical):
    ax = axes[int(idx / 4), idx % 4]
```

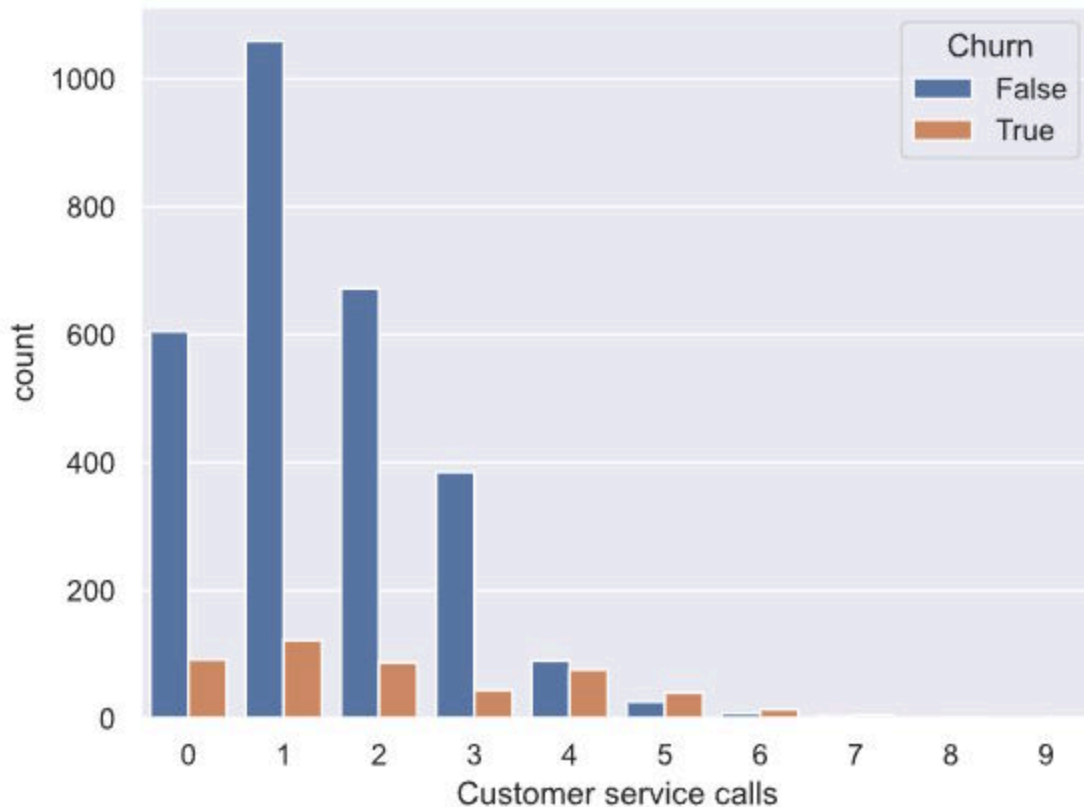
```
sns.boxplot(x="Churn", y=feat, data=df, ax=ax)
ax.set_xlabel("")
ax.set_ylabel(feat)
fig.tight_layout();
```



А на цих графіках ми бачимо як змінюється розподіл величин в колонках для кожного класу. І, як мінімум, дві відрізняються дуже суттєво. Це саме те, що ми шукаємо.

Бачимо, що графіки для кількості дзвінків в підтримку відрізняються. Крім того, цю колонку ми можемо розглянути як категоріальну, оскільки значень в ній не дуже багато.

```
sns.countplot(x="Customer service calls", hue="Churn", data=df);
```



Бачимо сутєву різницю. Більшість користувачів, що залишилися із компанією, мали до трьох дзвінків за цей період, а ті, що припинили користуватись послугами, дзвонили більше разів

Для візуалізації багатовимірних даних можна використати метод t-SNE

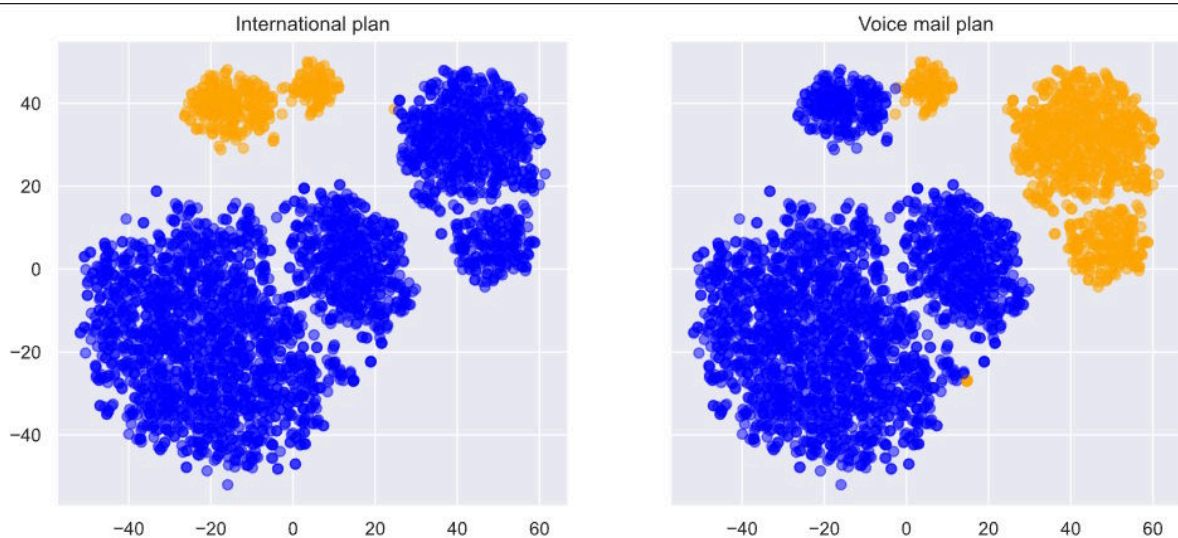
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
tsne = TSNE(random_state=17)
tsne_repr = tsne.fit_transform(X_scaled)
```

І, для прикладу, покажемо як згрупувались точки відносно двох категоріальних колонок

```
_, axes = plt.subplots(1, 2, sharey=True, figsize=(12, 5))

for i, name in enumerate(["International plan", "Voice mail plan"]):
```

```
axes[i].scatter(
    tsne_repr[:, 0],
    tsne_repr[:, 1],
    c=df[name].map({"Yes": "orange", "No": "blue"}),
    alpha=0.5,
)
axes[i].set_title(name);
```



Це не розподіл таргету, але ми розуміємо, що користувачі, як мають міжнародний тариф і голосову пошту виокремились в явні групи

Це лише невелика частина графіків, які можна побудувати при візуалізації датасету. Основна мета візуалізації - виявлення прихованих закономірностей в даних, та презентація результатів. І чим краще зроблена візуалізація, тим простіше буде на наступному етапі роботи із даними.

▼ Побудова нових змінних вручну

Досить часто виникає ситуація, досліднику треба збільшити кількість колонок, але нові дані відсутні. Базуючись на розумінні процесів, які описує датасет можна додати нові фічі. Так, якщо мова йде про продажі товару, логічно уявити, що подажі будуть залежати від пори року, або від дня тижня. Якщо це про відвідування ресторанів, то матимуть вплив такі

показники, як робочий чи вихідний день, час доби - ранок, день, вечір, ніч. Аналогічно, поведінка користувача в інтернеті буде мати певні цикли. Методами pandas це може бути реалізовано досить легко

```
df['hour'] = df['Time'].dt.hour.astype(float)
df['dayofweek'] = df['Time'].dt.dayofweek.astype(str)
df['quarter'] = df['Time'].dt.quarter.astype(str)
df['month'] = df['Time'].dt.month.astype(str)
df['year'] = df['Time'].dt.year.astype(str)
df['dayofyear'] = df['Time'].dt.dayofyear.astype(str)
df['dayofmonth'] = df['Time'].dt.day.astype(str)
df['weekofyear'] = df['Time'].dt.weekofyear.astype(str)
```

Категоріальні значення в колонках можуть бути перетворені в бінарні значення в кількох колонках, або цифрові значення, за допомогою енкoderів

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
le.fit(["paris", "paris", "tokyo", "amsterdam"])

enc = preprocessing.OneHotEncoder()
X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
enc.fit(X)

poly = preprocessing.PolynomialFeatures(2)    #2 is degree of new feature
poly.fit_transform(X)
```

В бібліотеці category_encoders зібрана певна кількість енкoderів. Розглянемо деякі з них. Імпортуємо

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import pandas as pd
```

```

from category_encoders.ordinal import OrdinalEncoder
from category_encoders.woe import WOEEncoder
from category_encoders.target_encoder import TargetEncoder
from category_encoders.sum_coding import SumEncoder
from category_encoders.m_estimate import MEstimateEncoder
from category_encoders.leave_one_out import LeaveOneOutEncoder
from category_encoders.helmert import HelmertEncoder
from category_encoders.cat_boost import CatBoostEncoder
from category_encoders.james_stein import JamesSteinEncoder

```

Найбільш поширений - OneHotEncoder. Він створює таку кількість нових колонок, скільки значень катеогріальної змінної є в колонці. Для кожного рядка він простваляє 1 в колонці, що відповідає значенню катеогрії, і 0 - в інших колонках

```

enc = OneHotEncoder(handle_unknown='ignore')
X = [['Male', 1], ['Female', 3], ['Female', 2]]
enc.fit(X)

```

```

[ ] enc.transform([['Female', 1], ['Male', 4], ['Male', 3], ['Male', 2]]).toarray()

array([[1., 0., 1., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 1., 0., 0., 1.],
       [0., 1., 0., 1., 0.]])

```

Перші дві колонки відповідають 'Female'/'Male', останні три - значенням 1/2/3. Звертаємо увагу, що числа 4, не було в датасеті при тренуванні енкодера, тому у відповідному рядку всі три значення - нулі

Неоднозначний, але також часто використовується LabelEncoder

```

le = LabelEncoder()
le.fit([1, 2, 2, 6])

```

```
le.classes_  
>> array([1, 2, 6])  
le.transform([1, 1, 2, 6])  
>>array([0, 0, 1, 2])
```

Цей енкодер кожному категоріальному значенню в колонці ставить у відповідність послідовні числа 0, 1, 2... Якщо категорій всього 2 (наприклад в таргеті), то цей енкодер можна викорисовувати без застережень. Якщо більше - слід мати на увазі, що ми отримуємо числові значення замість категорій. Модель буде сприймати це саме як числа і намагатиметься описати цю залежність. Хоча якщо значення категорії, наприклад, назви міст, то навряд чи ми можемо сказати, що Київ менше Харкова, а Одеса більше Львову.

Існує цілий клас енкодерів, які проставляють значення в залежності від впливу саме цього значення категорії на таргет

```
TE_encoder = TargetEncoder()  
df_te = TE_encoder.fit_transform(df[['sex']], df[['rings']])
```

```
df_te.head()
```

	sex
0	10.705497
1	10.705497
2	11.129304
3	10.705497
4	7.890462

Це більш релевантно, ніж LabelEncoder, і не треба турбуватись про озподіл значень вздовж числової вісі. З іншого боку, безпосереднього зв'язку між категоріальною змінною і таргетом може не існувати. І цей тип енкодера створить такий зв'язок. Як рекомендація - після застосування цього енкодера слід перевірити кореляцію з іншими колонками

Існують також енкодери, які замінюють категоріальне значення на вектор. Це може бути як бінарний вектор (щось типу LabelEncoder + представлення у двійковій системі), або може енкодер може натренувати векторне представлення категоріальних значень

Ще одним джерелом для створення нових колонок може бути агрегація даних. Часто дасет складається не з однієї таблиці, а з кількох. Можна агрегувати значення і додавати агреговані значення з однієї таблиці в іншу. Аналогічно, можна просто джойнити таблиці, збагачуючи дані однієї таблиці даними з іншої

Можна сказати, що створення нових колонок в датасеті, це свого роду мистецтво. І від того, як ви опануєте це мистецтво буде залежати результат моделювання

▼ Побудова нових змінних за допомогою бібліотек

▼ Бібліотека FeatureTools

Посилання:

Home

Featuretools uses DFS for automated feature engineering. You can combine your raw data with what you know about your data to build meaningful features for machine learning and predictive modeling.

<https://www.featuretools.com/>

Існують спеціальні бібліотеки, що дозволяють автоматично, майже без участі інженера, створювати нові колонки даних. Однією із таких бібліотек є FeatureTools

Послідовність роботи насувна. Завантажуємо дані в таблиці

```
# Read in the data
clients = pd.read_csv('data/clients.csv', parse_dates = ['joined'])
loans = pd.read_csv('data/loans.csv', parse_dates = ['loan_start', 'loan_end'])
```

```
payments = pd.read_csv('data/payments.csv', parse_dates
= ['payment_date'])
```

Сворюємо спеціальний об'єкт

```
es = ft.EntitySet(id = 'clients')
```

Додаємо дані з таблиць в створений об'єкт

```
# Create an entity from the client dataframe
# This dataframe already has an index and a time index
es = es.entity_from_dataframe(entity_id = 'clients', da
taframe = clients,
                                index = 'client_id', time
_index = 'joined')
# Create an entity from the loans dataframe
# This dataframe already has an index and a time index
es = es.entity_from_dataframe(entity_id = 'loans', data
frame = loans,
                                variable_types = {'repai
d': ft.variable_types.Categorical},
                                index = 'loan_id',
                                time_index = 'loan_star
t')
# Create an entity from the payments dataframe
# This does not yet have a unique index
es = es.entity_from_dataframe(entity_id = 'payments',
                                dataframe = payments,
                                variable_types = {'misse
d': ft.variable_types.Categorical},
                                make_index = True,
                                index = 'payment_id',
                                time_index = 'payment_dat
e')
```

Пропишуємо зв'язки між таблицями

```

# Relationship between clients and previous loans
r_client_previous = ft.Relationship(es['clients']['client_id'],
                                     es['loans']['client_id'])

# Add the relationship to the entity set
es = es.add_relationship(r_client_previous)

# Relationship between previous loans and previous payments
r_payments = ft.Relationship(es['loans']['loan_id'],
                              es['payments']['loan_id'])

# Add the relationship to the entity set
es = es.add_relationship(r_payments)

```

Описуємо які перетворення треба зробити

```

# Create new features using specified primitives
features, feature_names = ft.dfs(entityset = es, target_entity = 'clients',
                                  agg_primitives = ['mean', 'max', 'percent_true', 'last'],
                                  trans_primitives = ['years', 'month', 'subtract', 'divide'])

```

Запускаємо процес синтезу нових колонок

```

# Perform deep feature synthesis without specifying primitives
features, feature_names = ft.dfs(entityset=es, target_e

```

```
ntity='clients',  
max_depth = 2)
```

Для даного прикладу було автоматично згенеровано 90 нових колонок. Вручну це могло б зайняти кілька днів. За допомогою цієї бібліотеки - не більше дня

Повний опис і цей приклад можна знайти в додакових матеріалах